

Multi-Agent AI Support Ticket System

Group Project 1 — A2A Communication, RAG, and Browser Automation

Group 2: Parth Auti, Hasti Bhalodi, Medha Jani, Kento Hopkins

September 28, 2026

1. What the System Does

Our system takes a description of an IT problem written in plain English and files a support ticket for it. Nobody has to pick the category or write the resolution notes by hand. It does this with two agents that talk to each other over HTTP.

The Requester Agent is in charge of the overall job. It takes the user's request, sends it to the Specialist as a task, waits for an answer by polling, works out what goes in each box on the support form, uses Playwright to fill in the form and submit it, and then checks the confirmation page to make sure the right thing was filed.

The Specialist Agent is the one that actually knows things. It searches the seven support documents we were given, pulls out the passages that look relevant, and passes those to an LLM to write an answer. It sends back a ticket category, some resolution notes, and the names of the files it used.

The important part is that the Specialist's answer is not just printed out. The category it picks decides which option gets selected in the dropdown, and the resolution it writes is the text that goes into the resolution box. If the Specialist returns a category that the form has no option for, the Requester stops and does not submit anything.

2. Architecture and Workflow

Here is what happens when a request comes in, step by step:

1. The user's request comes in through the command line and goes to `handle_request()`.
2. The Requester works out that it needs two things, a category and a resolution, and sends the issue text to the Specialist with `POST /tasks`.
3. The Specialist saves the task, starts a worker thread, and replies straight away with a task ID and the status `submitted`.
4. The Requester keeps calling `GET /tasks/{id}` to check on it.
5. The worker sets the task to `working` and runs the RAG pipeline.
6. The pipeline searches the knowledge base and generates an answer.
7. The task becomes `completed` and the next poll gets the result back.

8. The Requester turns the category into a dropdown value and Playwright fills in the form and submits it.
9. The Requester reads the confirmation page and checks it matches what the Specialist said.

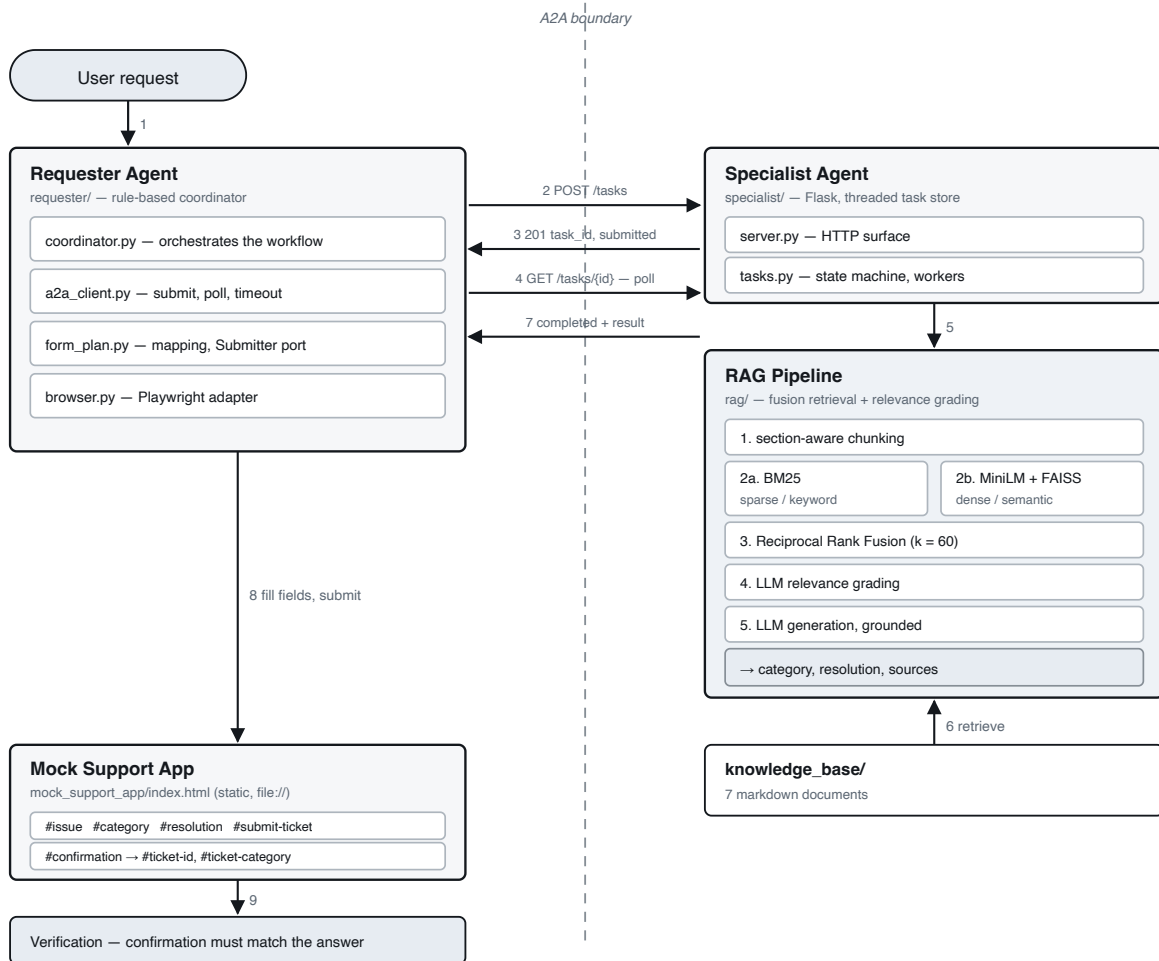


Figure 1: System architecture and request flow. The dashed line is a real process boundary, since the two agents run as separate processes and only talk over HTTP.

Why we split it up this way

We kept three boundaries strict, and all three ended up mattering during the project.

The Specialist does not know that a web form exists. It gives back whatever category the knowledge base uses, and the Requester is the one that turns that into a dropdown value. The task store does not know that RAG exists either, because the pipeline gets passed in as a function. That turned out to be useful, since we replaced the whole retrieval layer partway through the project and never had to touch the server or the state machine. And the coordinator does not know Playwright exists, because it goes through a

Submitter interface. That let the browser work be written at the same time as everything else instead of after it.

3. The A2A Protocol

We wrote the protocol down in a contracts file before writing any code, and agreed that nobody would change it without telling the group. This is probably the main reason four people could work on four different parts at once without breaking each other's code.

Endpoints

Endpoint	What it does
POST /tasks	Submits a task. Replies with 201, a task ID, and the status submitted. It does not wait for the work to finish.
GET /tasks/{id}	Gives the current status. Includes result once the task is done, or error if it failed. Returns 404 if the ID is not recognised.
GET /health	Checks the server is up.

Task states

State	Finished?	Meaning
submitted	No	Accepted but not started yet.
working	No	The RAG pipeline is running.
completed	Yes	The result is ready.
failed	Yes	Something went wrong; there is an error instead of a result.

What the messages look like

[illegible]

```
"sources": ["password_reset.md"],  
"confidence": 0.82}}
```

Error codes

We gave the errors codes instead of just returning a message, so the Requester can tell them apart and react differently. `NO_RELEVANT_CONTEXT` means the knowledge base had nothing useful and no ticket should be filed at all. `LLM_ERROR` means the model call itself broke, which is something a retry might fix. `UNKNOWN_TASK` and `BAD_REQUEST` are problems with the request. `TIMEOUT` and `SPECIALIST_UNREACHABLE` come from the client side rather than the server.

Decisions we made

The biggest one is that submitting a task does not wait for the answer. The work runs on a background thread, so `POST /tasks` comes back while retrieval is still going. If we had made it wait, the task IDs and the polling loop would not really be doing anything. One of our tests checks this on purpose by submitting with a delay turned on and making sure the reply still comes back in under a second.

We also had to make the acknowledgment always say `submitted`. What we found was that the worker thread could sometimes move the task to `working` before the `201` reply had even been sent, so a submission could come back with a status that did not match our own contract. Now the reply is a fixed snapshot of "we accepted this" no matter what the stored task has already changed to.

The last one caused us the most trouble. Originally we wrapped the user's issue in instruction text before sending it, something like "Using the support knowledge base, identify the correct category...", and sent that whole string as the question. The problem is that the Specialist uses the question directly as its search query, so all that instruction wording ended up in the search and drowned out the actual problem description. Three of the five test cases stopped working. We moved the prompt wording into the RAG layer where it belongs, and now the protocol just carries the user's own words plus a separate `needs` field saying what we want back.

4. The RAG Pipeline

The pipeline runs in five stages over the seven documents.

1. **Load and chunk.** Each document gets split up by its markdown sections. We only build chunks out of the sections that describe problems, which are Description, Symptoms and When to Use, rather than the whole file.
2. **Build two indexes.** One BM25 keyword index and one dense index using `all-MiniLM-L6-v2` embeddings stored in FAISS, both over the same chunks. The index gets cached to disk with a hash of the documents, so it only gets rebuilt if the documents change.

3. **Fuse the results.** Both indexes produce their own ranking, and we combine the two with Reciprocal Rank Fusion using $k = 60$, keeping the top eight.
4. **Grade them.** We ask the LLM in one call whether each of those eight passages is actually relevant, and throw away the ones that are not. If none survive, the task fails with `NO_RELEVANT_CONTEXT`.
5. **Generate.** The category is read straight out of the winning document's own Ticket Category section, and the LLM writes one to three sentences of resolution notes using only that document's Resolution text, at temperature 0.

The advanced technique we picked

We went with fusion retrieval, which means running keyword and semantic search separately and combining the two rankings with Reciprocal Rank Fusion, plus LLM relevance grading before anything gets generated. Neither is in the foundational group of techniques.

We picked it because of what these particular documents look like. All seven are written to the same template, with the same Description, Symptoms, Troubleshooting and Resolution headings, and a lot of the same sentences. "Restart the computer if appropriate" shows up in four of them. Embeddings pick up on all that shared structure, which makes documents look more alike than they are, and getting `network.md` mixed up with `hardware.md` means the wrong option gets picked in the dropdown. BM25 keeps exactly the signal that embeddings lose, which is the literal words "Wi-Fi" and "keyboard". Putting both together keeps the keyword precision without giving up the semantic side that connects something like "cannot log in" to "account locked".

We added the grading step for a different reason. Without it, a question the documents do not cover still gets the closest chunk back, and the LLM will happily write a confident answer from it. Grading gives the pipeline a way to say that nothing here actually answers this, which is what makes our `NO_RELEVANT_CONTEXT` case a real failure path instead of one we just claim exists.

Some smaller decisions

- We only index the problem sections. When we included the Resolution sections as well, every document started looking similar, because the resolutions are all written in the same style.
- The category is copied out of the document rather than generated. It has to match one of a fixed set of options exactly, so there is no reason to let a model reword it. Only the resolution notes get generated.
- Grading is one call with all eight passages numbered, rather than eight separate calls, to keep it faster and cheaper.
- The source filenames always come back with the answer, so any result can be traced to the documents behind it.

5. The Playwright Part

The mock application is a plain HTML file with no backend, so we load it with a `file://` URL built from wherever the repository is. The submitter then does this:

1. Starts Chromium and opens the page.
2. Fills `#issue` with the user's original request.
3. Picks `#category` using the option value we worked out from the Specialist's category.
4. Fills `#resolution` with the generated notes.
5. Clicks `#submit-ticket`.
6. Waits for `#confirmation:not(.hidden)`. If it never shows up, we read whatever is in `#error` on the page and fail with `VERIFICATION_FAILED` carrying that message.
7. Reads `#ticket-id`, `#ticket-category` and `#ticket-resolution`, and checks the ticket ID is five digits.
8. Closes the browser in a `finally` block so it always shuts down.

Checking the result properly

A confirmation box showing up only tells you the form accepted something. So the coordinator also compares the category and resolution printed on the confirmation page against what the Specialist actually returned, and fails the run if they do not match. That way a ticket filed under the wrong category cannot be counted as a success.

Mapping categories, and knowing when to stop

The Specialist gives back categories in the wording the knowledge base uses, so the Requester has to map those onto the four values the dropdown accepts:

Knowledge base category	Form option
Account Access	account_access
Hardware	hardware
Software	software
Network	network
Email	no option on the form
Security	no option on the form

This is where we hit something we did not expect. The knowledge base has documents for Email and Security, and one of the provided test cases is an email problem with "Email" as its expected category, but the form's dropdown does not have an Email option. Rather than quietly picking whichever category is closest, we made the Requester raise `UNMAPPABLE_CATEGORY` and refuse to submit. The mapping is checked before the browser even starts, so this fails quickly. Our reasoning was that a ticket filed under a category we already know is wrong is worse than no ticket at all.

6. Error Handling

What happens	When	What the system does
<code>NO_RELEVANT_CONTEXT</code>	None of the retrieved passages were graded relevant.	Task ends as <code>failed</code> . The Requester reports it and never opens a browser.
<code>TIMEOUT</code>	The task was still running when the client gave up waiting.	Stops polling and reports how long it waited and how many times it checked.
<code>UNMAPPABLE_CATEGORY</code>	The category has no matching option on the form.	Refuses before launching the browser, and says which category it got and which ones exist.
<code>LLM_ERROR</code>	The model call failed or gave back something we could not parse.	Any exception in the worker turns into a failed task instead of crashing the server.
<code>VERIFICATION_FAILED</code>	The confirmation never appeared, or did not match.	Reported as a failure, including whatever error the page itself displayed.
<code>SPECIALIST_UNREACHABLE</code>	The Specialist is not running.	Reported immediately instead of hanging.
<code>UNKNOWN_TASK</code>	The task ID does not exist.	404 with the code, and the client stops polling.

Two things made all of this easier to handle. The coordinator never throws an exception. Every failure comes back as a normal return value with an error code on it, which is why we can demonstrate any of the cases above on demand. And any unexpected crash inside a worker gets caught and turned into a failed task, so a bug in retrieval breaks one ticket rather than taking the whole agent down.

7. Results

We measured two things. The first looks at retrieval on its own, and the second runs the whole system through a real browser.

Retrieval

For each of the five test cases we checked which document each retriever put first, and whether that document has the expected ticket category on it. There is no LLM involved in this measurement, which

we did on purpose so that the numbers only reflect the retrieval change and not anything the grading or generation steps might add.

Retriever	Right document ranked first		
Baseline keyword matching	4 out of 5		
Fusion retrieval (BM25 + dense + RRF)	5 out of 5		

Test case	Expected	Baseline picked	Fusion picked
Forgot password, cannot log in	Account Access	account_access.md	password_reset.md
Laptop will not connect to Wi-Fi	Network	hardware.md (wrong)	network.md
Keyboard has stopped working	Hardware	hardware.md	hardware.md
Cannot install software	Software	software.md	software.md
Company email not syncing	Email	email.md	email.md

The one case that changed is the one we were hoping would change. For "My laptop won't connect to Wi-Fi", the baseline puts `hardware.md` first, because the word "laptop" appears in the hardware symptoms and outweighs the single mention of "Wi-Fi". Fusion retrieval puts `network.md` first instead, because the BM25 half keeps the word that actually distinguishes the two documents, and RRF pushes up a document that both retrievers agree on over one that only keyword overlap liked.

The password case is worth a mention too. Both retrievers end up with the right category, but fusion picks `password_reset.md` where the baseline picks `account_access.md`. Both documents say the category is Account Access so the ticket comes out the same either way, but the more specific document is the better match, since it is the procedure that actually covers a forgotten password.

Running the whole thing

Running all five cases through the full system, meaning both agents, the protocol, and Playwright driving a real Chromium window against the form, gives four submitted tickets with valid five digit IDs and the right categories. The fifth one stops at `UNMAPPABLE_CATEGORY`, because the knowledge base correctly says it is an Email issue and the form has no Email option. We are counting that as the system working rather than failing, since the only alternative would have been filing it under a category we knew was wrong.

8. Reflection

What went well

Writing the protocol and the result format down before we started coding was the best decision we made. It meant all four of us could work at the same time, each one testing against a fake version of everyone else's part, and things mostly just fit together when we joined them up. The two plug-in points, where the RAG pipeline goes in as a function and the browser sits behind an interface, are what made that actually work rather than just sounding good on paper. We swapped out the entire retrieval layer partway through and did not have to change a line in the agent that calls it.

Having the coordinator return a result object instead of throwing exceptions also helped more than we expected. It meant every failure case was something we could trigger and show on demand instead of just describing.

What did not go well

Our first version put the prompt wording into the message we sent between agents, which quietly broke three of the five test cases because that wording ended up in the search query. It took a while to work out what was happening, since nothing crashed, the answers just got worse. The lesson was that the search query and the prompt you give the model are two different things and should not live in the same place.

We also had a state in our protocol that technically existed but could never be seen. Our artificial delay was in the wrong spot, before the switch to `working` instead of after it, so tasks jumped from `submitted` straight to `completed` and nobody polling would ever catch the middle state. The code looked fine and was still wrong. That is what made us write tests that check each state can actually be observed, not just that it is defined somewhere.

How well RAG worked

The difference between the two retrievers came out to one case in five. That is encouraging but we do not want to read too much into it, because five test cases is nowhere near enough to tune anything against, and one result flipping could easily be luck. What it does show is that the problem we predicted just by reading the documents is the same one fusion retrieval ended up fixing.

More generally, how well retrieval works on this set of documents depends much more on how similar they are to each other than on how much text there is. With seven documents all written to the same template, the hard part is telling them apart rather than finding them, which is why mixing keyword and semantic search beat either one alone, and why choosing which sections to index mattered more than choosing a chunk size. A much bigger knowledge base would probably change that balance.

Limitations

- Tasks are only kept in memory, so restarting the server loses all of them.
- There are no retries, so one bad LLM call fails a ticket that would otherwise have been fine.

- There is no way to cancel a task, so if the Requester times out the work carries on and finishes into a result nobody reads.
- The confidence number is just a retrieval score. It is not a real probability and should not be treated as one.
- The Requester only knows about one Specialist, and there is no way to discover others.

What we would do next

- Add retries with backoff, and tell the difference between a temporary failure and a task that genuinely cannot be done.
- Store tasks somewhere persistent, and add cancellation so abandoned work stops.
- Have the Specialist publish a description of itself so the Requester could find agents and choose between them, which is what we would need before adding a second specialist.
- Build a proper evaluation set with more than five cases, since five is too few to trust a threshold tuned against it.
- Decide what should actually happen with the Email and Security categories, either by adding them to the form or by defining an escalation path, instead of only refusing.

9. Who Did What

Member	Work
Parth Auti	Requester Agent and Specialist Agent (steps 1 and 2), the contracts file, and the project structure.
Hasti Bhalodi	The A2A protocol (step 3), including the task lifecycle, the state machine, the error codes, and the protocol tests.
Medha Jani	The RAG pipeline and advanced technique (step 4): chunking, both indexes, Reciprocal Rank Fusion, grading, and generation.
Kento Hopkins	Playwright automation and verification (steps 5 and 6), and the browser side of the error handling.

Appendix: How to Run It

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
playwright install chromium
cp .env.example .env          # add GROQ_API_KEY for the full pipeline

# terminal 1
python -m specialist.server --delay 1
```

```
# terminal 2
python -m requester.cli --all --browser
python -m unittest test_a2a_protocol -v
```

To see the timeout case, start the server with `--delay 999` and run the Requester with `--timeout 3`. To see `NO_RELEVANT_CONTEXT`, ask it something the documents do not cover, like "What is the capital of France?".